Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Наивный поиск образца в строке

Группа: М8О-211БВ-25

Студент: Самсонов И.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 15.10.26

Москва, 2026

Постановка задачи

Вариант 18. Найти образец в строке наивным алгоритмом.

Цель работы - получить практические навыки создания и синхронизации потоков ОС, реализовать последовательную и параллельную версии поиска и исследовать зависимость времени, ускорения и эффективности от входных данных и числа потоков.

Программа написана на C11. Для создания и ожидания потоков используются POSIX Threads. Максимальное общее число потоков задаётся ключом -t. Для обеих версий измеряется длительность работы; число потоков дополнительно демонстрируется стандартным средством ОС.

Общий метод и алгоритм решения

Наивный алгоритм последовательно прикладывает образец к каждой допустимой позиции текста. Символы сравниваются слева направо до первого несовпадения либо конца образца. При совпадении всех символов сохраняется позиция начала. Перекрывающиеся совпадения учитываются; позиции отсчитываются от нуля в байтах.

Пусть n - длина текста, m - длина непустого образца. При $m \le n$ число допустимых начал $L = n - m + 1$. В худшем случае последовательная версия выполняет $O(Lm)$ сравнений. При $m > n$ или пустом тексте совпадений нет. Пустой образец отклоняется с сообщением об ошибке.

При -t 1 весь диапазон [0, L) обрабатывает главный поток, дополнительные потоки не создаются. В параллельной версии при $L > 0$ число участников $p = \min(N, L)$, где N - заданный предел. При $L = 0$ поиск не выполняется и остаётся один главный поток. Создаются $p - 1$ дополнительных потоков, последнюю задачу выполняет главный.

Диапазон начал делится на p частей. Каждый участник получает $q = L \operatorname{div} p$ позиций; первые $r = L \bmod p$ участников получают ещё по одной. Например, $L = 10$ и $p = 4$ дают размеры 3, 3, 2, 2. Чтение общего текста не ограничивается границей части, поэтому совпадения, пересекающие границы диапазонов, не теряются.

Текст и образец доступны только для чтения. Каждый участник записывает отдельный счётчик и свои элементы массива matches. Главный поток читает результаты после pthread_join, суммирует счётчики и печатает позиции по возрастанию. Так как области записи не пересекаются, мьютекс для этих данных не требуется. Потоки создаются через pthread_create, а ожидание выполняется через pthread_join [2, 3].

Программа состоит из структуры Task и трёх функций: search выполняет поиск на своём участке, read_file читает обычный файл, main разбирает аргументы, распределяет работу и выводит результат. Ошибки ввода, памяти и создания потоков проверяются.

Код программы

main.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    const char *text;
    const char *pattern;
    size_t pattern_length;
    size_t begin;
    size_t end;
    unsigned char *matches;
    size_t count;
} Task;

void *search(void *arg)
{
    Task *task = arg;
    task->count = 0;
    for (size_t i = task->begin; i < task->end; i++) {
        size_t j = 0;
        while (j < task->pattern_length && task->text[i + j] == task->pattern[j]) {
            j++;
        }
        if (j == task->pattern_length) {
            task->count++;
            if (task->matches != NULL) {
                task->matches[i] = 1;
            }
        }
    }
    return NULL;
}

char *read_file(const char *name, size_t *length)
{
    FILE *file = fopen(name, "rb");
    if (file == NULL) {
        perror(name);
        return NULL;
    }
    if (fseek(file, 0, SEEK_END) != 0) {
        fclose(file);
        return NULL;
    }
    long size = ftell(file);
    if (size < 0 || fseek(file, 0, SEEK_SET) != 0) {
        fclose(file);
        return NULL;
    }
    /* Для пустого файла тоже нужен ненулевой указатель. */
    char *text = malloc(size > 0 ? (size_t)size : 1);
    if (text == NULL) {
        fclose(file);
        return NULL;
    }
    size_t bytes = fread(text, 1, (size_t)size, file);
    int error = ferror(file);
    int close_error = fclose(file);
    if (bytes != (size_t)size || error || close_error != 0) {
        free(text);
        return NULL;
    }
```

```
    *length = bytes;
    return text;
}

int main(int argc, char *argv[])
{
    int limit = 0, quiet = 0, option;
    const char *filename = NULL;
    while ((option = getopt(argc, argv, "t:f:q")) != -1) {
        switch (option) {
        case 't': {
            char *end;
            errno = 0;
            long value = strtol(optarg, &end, 10);
            if (errno || end == optarg || *end != '\0' || value < 1 || value > INT_MAX) {
                fprintf(stderr, "Число потоков должно быть положительным целым\n");
                return 1;
            }
            limit = (int)value;
            break;
        }
        case 'f': filename = optarg; break;
        case 'q': quiet = 1; break;
        default: return 1;
        }
    }
    if (limit == 0 || argc - optind != (filename != NULL ? 1 : 2)) {
        fprintf(stderr, "Запуск: %s -t N [-q] ТЕКСТ ОБРАЗЕЦ\n", argv[0]);
        fprintf(stderr, "Или:    %s -t N [-q] -f ФАЙЛ ОБРАЗЕЦ\n", argv[0]);
        return 1;
    }
    const char *pattern = argv[argc - 1];
    size_t m = strlen(pattern);
    if (m == 0) {
        fprintf(stderr, "Образец не должен быть пустым\n");
        return 1;
    }
    char *file_text = NULL;
    const char *text;
    size_t n;
    if (filename != NULL) {
        file_text = read_file(filename, &n);
        if (file_text == NULL) {
            fprintf(stderr, "Не удалось прочитать файл\n");
            return 1;
        }
        text = file_text;
    } else {
        text = argv[optind];
        n = strlen(text);
    }

    size_t positions = m <= n ? n - m + 1 : 0;
    int p = limit;
    if (positions == 0) p = 1;
    else if (positions < (size_t)p) p = (int)positions;
    unsigned char *matches = NULL;
    if (!quiet && positions > 0) {
        matches = calloc(positions, 1);
        if (matches == NULL) {
            free(file_text);
            fprintf(stderr, "Не хватает памяти\n");
            return 1;
        }
    }
    struct timespec start, finish;
    if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
        perror("clock_gettime");
        free(matches);
        free(file_text);
        return 1;
    }
```

```
    Task *tasks = calloc((size_t)p, sizeof(Task));
    pthread_t *threads = calloc((size_t)p, sizeof(pthread_t));
    if (tasks == NULL || threads == NULL) {
        free(tasks); free(threads); free(matches); free(file_text);
        fprintf(stderr, "Не хватает памяти\n");
        return 1;
    }
    size_t part = positions / (size_t)p;
    size_t extra = positions % (size_t)p;
    size_t begin = 0;
    for (int i = 0; i < p; i++) {
        size_t length = part + ((size_t)i < extra ? 1 : 0);
        /* Делим позиции начала, чтобы не терять совпадения на границах. */
        tasks[i] = (Task){text, pattern, m, begin, begin + length, matches, 0};
        begin += length;
    }
    int created = 0, failed = 0;
    for (int i = 0; i < p - 1; i++) {
        int error = pthread_create(&threads[i], NULL, search, &tasks[i]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            failed = 1;
            break;
        }
        created++;
    }
    /* Главный поток тоже ищет. При p = 1 это последовательная версия. */
    if (!failed) search(&tasks[p - 1]);
    for (int i = 0; i < created; i++) {
        int error = pthread_join(threads[i], NULL);
        if (error != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(error));
            exit(1);
        }
    }
    size_t count = 0;
    for (int i = 0; i < p; i++) count += tasks[i].count;
    free(tasks);
    free(threads);
    if (clock_gettime(CLOCK_MONOTONIC, &finish) != 0) {
        perror("clock_gettime");
        failed = 1;
    }
    if (!failed) {
        double ms = (finish.tv_sec - start.tv_sec) * 1000.0
                  + (finish.tv_nsec - start.tv_nsec) / 1000000.0;
        if (!quiet) {
            for (size_t i = 0; i < positions; i++) {
                if (matches[i]) printf("%zu ", i);
            }
            printf("\n");
        }
        printf("Совпадений %zu\nПотоков %d\nВремя %.6f мс\n", count, p, ms);
        if (fflush(stdout) != 0 || ferror(stdout)) failed = 1;
    }
    free(matches);
    free(file_text);
    return failed;
}
```

Протокол работы программы

Сборка выполнена Apple clang 21.0.0 с оптимизацией -O2 и предупреждениями. Основной способ сборки - make; эквивалентная команда приведена ниже. Ключ -f читает текст из файла, -q оставляет только число совпадений и статистику.

```
cc -std=c11 -O2 -Wall -Wextra -Wpedantic \
    main.c -o main -pthread
```

Проверка последовательного и параллельного поиска

```
$ ./main -t 1 ababa aba
0 2
Совпадений 2
Потоков 1
Время 0.000000 мс

$ ./main -t 4 aaaa aa
0 1 2
Совпадений 3
Потоков 3
Время 0.028000 мс

$ ./main -t 2 abc z

Совпадений 0
Потоков 2
Время 0.022000 мс
```

Совпадения в позициях 0 и 2 для ababa/aba и 0, 1, 2 для aaaa/aa подтверждают учёт перекрытий. При пределе 4 на этих коротких входах участников только три. Нулевое время на очень коротком поиске связано с разрешением таймера и не означает отсутствия вычислений.

При проверке выполнены 274 сравнения с независимым поиском: проверены границы участков, перекрытия, 200 случайных примеров, бинарные и пустые файлы, UTF-8. Ещё 16 запусков с неверными аргументами или путями завершились ошибкой. Дополнительно выполнена сборка с AddressSanitizer и UndefinedBehaviorSanitizer; ошибок памяти не обнаружено.

Демонстрация числа потоков средствами ОС

Для наблюдения использован отдельный длительный запуск: текст из 16 000 000 символов a и образец a × 1023 + b. Все 1024 символа образца сравниваются в каждой допустимой позиции; совпадений нет. Четыре потока наблюдаются командой macOS ps -M -p PID. Искусственных задержек в алгоритме нет.

```
# В demo.txt: 16 000 000 букв a; в pattern: 1023 буквы a и b
./main -t 4 -q -f demo.txt "$pattern" > demo_result.txt &
lab_pid=$!
ps -M -p "$lab_pid"
wait "$lab_pid"
cat demo_result.txt
```

Для длительного поиска получен следующий вывод ps. Приведены столбцы до COMMAND; длинная командная строка опущена. Четыре строки под заголовком соответствуют главному и трём дополнительным потокам.

```
USER   PID   TT   %CPU STAT PRI     STIME     UTIME
ivan 42035   ??   13.9 R    31T   0:00.00   0:00.10
     42035        12.0 R    31T   0:00.00   0:00.11
     42035        12.0 R    31T   0:00.00   0:00.11
     42035        12.4 R    31T   0:00.00   0:00.11
Совпадений 0
Потоков 4
Время 1048.490000 мс
```

Это отдельная демонстрация потоков. Её время не включается в таблицы исследования.

Трассировка системных вызовов

Для проверки работы потоков выполнен отдельный запуск под strace в Linux: Ubuntu 24.04 в Docker, aarch64, GCC 13.3.0, strace 6.8. Использован тот же main.c: текст из 16 000 000 букв a, образец из 127 букв a и буквы b, предел -t 4. Эта трассировка не входит в замеры времени на macOS.

```
strace -f -o full.log ./main -t 4 -q -f input.txt "$pattern"
# При новом запуске можно сразу оставить только нужные вызовы:
strace -f -e trace=clone,clone3,futex,exit,exit_group,write \
    ./main -t 4 -q -f input.txt "$pattern"
```

Ключ -f включает наблюдение за создаваемыми потоками, -o сохраняет трассу, -e trace выбирает системные вызовы. Ниже приведён сокращённый фрагмент первого запуска. Адреса и длинные аргументы заменены на ...; номера потоков и результаты сохранены. Загрузка библиотек, выделение памяти и повторяющиеся ожидания опущены [6].

```
1085  clone3({...}, 88) = -1 ENOSYS (Function not implemented)
1085  clone(..., flags=CLONE_VM|...|CLONE_THREAD|..., ...) = 1086
1085  clone(..., flags=CLONE_VM|...|CLONE_THREAD|..., ...) = 1087
1085  clone(..., flags=CLONE_VM|...|CLONE_THREAD|..., ...) = 1088
1085  futex(..., FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 1086, NULL, ... <unfinished ...>
1086  exit(0)                           = ?
1085  <... futex resumed>)              = 0
1085  write(1, ..., 67) = 67
1085  exit_group(0)                     = ?
```

Первое число в строке - TID, то есть номер потока в Linux. Главный поток 1085 трижды вызывает clone и получает 1086, 1087 и 1088. Вместе с главным это четыре потока. CLONE_VM означает общую память, CLONE_THREAD - принадлежность одной группе потоков. Эти вызовы выполняет библиотека внутри pthread_create [7].

clone3 вернул -1 ENOSYS: в данном окружении этот вызов недоступен. После этого библиотека успешно перешла на clone. Это не ошибка алгоритма и не дополнительный созданный поток.

futex с ожидаемым значением 1086 связан с pthread_join: главный поток ждёт завершения потока 1086. Пометка unfinished означает, что вызов ещё не завершился; resumed - что его выполнение продолжилось. После exit(0) рабочего потока ожидание возвращает 0. Аналогичные ожидания для 1087 и 1088 в полном логе тоже присутствуют. Если поток уже завершился, pthread_join может обойтись без системного ожидания [6, 8].

write(1, ..., 67) записывает 67 байт результата в стандартный вывод: дескриптор 1 - stdout, значение после = - число записанных байт. exit_group(0) завершает весь процесс с кодом 0. pthread_create и pthread_join - функции библиотеки, поэтому strace показывает их системные вызовы clone и futex, а не эти имена [8].

Методика исследования

Эксперименты выполнены 15 октября 2026 года на macOS 26.6.2, архитектура arm64, 12 логических CPU. Использована оптимизированная сборка -O2 без санитайзеров. Последовательная база - отдельный запуск -t 1; параллельные конфигурации используют те же входные данные и режим -q.

Для каждого набора выполняются четыре конфигурации с числом участников меньше 12: 2, 4, 6, 8; одна конфигурация с 12 участниками; три конфигурации значительно выше числа CPU: 32, 128, 1024. Число допустимых позиций достаточно велико, поэтому фактическое число участников равно запрошенному пределу.

Входы имеют длины 100 000, 1 000 000 и 5 000 000 байт. Для каждой из 108 конфигураций выполнен один прогрев и пять измерений. В таблицах приведены медианы пяти измерений; всего выполнено 540 измерений. Порядок чисел потоков в повторах перемешивается с seed 18.

Использованы четыре набора. random_16 и random_128 - текст из псевдослучайных латинских букв и образцы a × 15 + b и a × 127 + b. Текст строится повторением случайного блока длиной 65 536 байт; отсутствие образца проверяется. many_matches_16 - текст из a и образец a × 16, совпадений n - 15. worst_case_128 - текст из a и образец a × 127 + b, совпадений нет; в каждой позиции выполняются все 128 сравнений.

Длительность вычисляется как разность двух показаний CLOCK_MONOTONIC. Время включает выделение задач, разбиение диапазонов, создание потоков, поиск, ожидание, суммирование и освобождение массивов задач и потоков. Чтение файла, выделение массива позиций и вывод исключены. В -q позиции не сохраняются, но число совпадений вычисляется и проверяется во всех запусках [4].

```
make
./main -t 1 -q -f text.txt aaaaaaaaaaaaaaab
./main -t 12 -q -f text.txt aaaaaaaaaaaaaaab
```

Для эксперимента меняются входной файл, образец и значение -t. На каждом повторе проверяется количество совпадений; для расчёта S и E используется последовательное время на том же входе.

Ускорение и эффективность

Ускорение $S(p) = T_1 / T_p$ показывает, во сколько раз выполнение с p участниками быстрее последовательного. T_1 - медиана времени последовательной версии, T_p - медиана времени параллельной версии на том же входе. Если $S < 1$, параллельная версия медленнее.

Эффективность $E(p) = S(p) / p$ показывает ускорение в расчёте на одного участника. Здесь p включает главный поток. В таблицах E выражена в процентах: $E\% = 100 \times T_1 / (p \times T_p)$. Значение 100% соответствует идеальному линейному ускорению; меньшие значения отражают накладные расходы и ограничения параллельного выполнения.

Случайный текст и образец длиной 16 байт

Таблица 1. Время исполнения, ускорение и эффективность

Длина текста байт	Число потоков	Время мс	Ускорение S	Эффективность %
100 000	1	0,069	1,000	100,00
100 000	2	0,069	1,000	50,00
100 000	4	0,115	0,600	15,00
100 000	6	0,105	0,657	10,95
100 000	8	0,108	0,639	7,99
100 000	12	0,104	0,663	5,53
100 000	32	0,259	0,266	0,83
100 000	128	1,034	0,067	0,05
100 000	1024	9,853	0,00700	0,0007
1 000 000	1	0,714	1,000	100,00
1 000 000	2	0,402	1,776	88,81
1 000 000	4	0,234	3,051	76,28
1 000 000	6	0,218	3,275	54,59
1 000 000	8	0,206	3,466	43,33
1 000 000	12	0,206	3,466	28,88
1 000 000	32	0,316	2,259	7,06
1 000 000	128	1,118	0,639	0,50
1 000 000	1024	9,984	0,072	0,0070
5 000 000	1	3,287	1,000	100,00
5 000 000	2	1,854	1,773	88,65
5 000 000	4	0,954	3,445	86,14
5 000 000	6	0,720	4,565	76,09
5 000 000	8	0,752	4,371	54,64
5 000 000	12	0,659	4,988	41,57
5 000 000	32	0,609	5,397	16,87
5 000 000	128	1,251	2,627	2,05
5 000 000	1024	9,666	0,340	0,03

Длина образца 16 байт. Для текста длиной 5 млн байт наименьшая медиана среди проверенных конфигураций получена при p = 32: 0,609 мс, ускорение 5,397.

Случайный текст и образец длиной 128 байт

Таблица 2. Время исполнения, ускорение и эффективность

Длина текста байт	Число потоков	Время мс	Ускорение S	Эффективность %
100 000	1	0,069	1,000	100,00
100 000	2	0,066	1,045	52,27
100 000	4	0,116	0,595	14,87
100 000	6	0,109	0,633	10,55
100 000	8	0,106	0,651	8,14
100 000	12	0,118	0,585	4,87
100 000	32	0,260	0,265	0,83
100 000	128	1,016	0,068	0,05
100 000	1024	9,888	0,00698	0,0007
1 000 000	1	0,666	1,000	100,00
1 000 000	2	0,403	1,653	82,63
1 000 000	4	0,264	2,523	63,07
1 000 000	6	0,222	3,000	50,00
1 000 000	8	0,209	3,187	39,83
1 000 000	12	0,207	3,217	26,81
1 000 000	32	0,303	2,198	6,87
1 000 000	128	1,083	0,615	0,48
1 000 000	1024	9,654	0,069	0,0067
5 000 000	1	3,286	1,000	100,00
5 000 000	2	1,858	1,769	88,43
5 000 000	4	0,951	3,455	86,38
5 000 000	6	0,713	4,609	76,81
5 000 000	8	0,747	4,399	54,99
5 000 000	12	0,657	5,002	41,68
5 000 000	32	0,591	5,560	17,38
5 000 000	128	1,235	2,661	2,08
5 000 000	1024	9,677	0,340	0,03

Длина образца 128 байт. Для текста длиной 5 млн байт наименьшая медиана среди проверенных конфигураций получена при p = 32: 0,591 мс, ускорение 5,560.

Повторяющийся текст и много совпадений

Таблица 3. Время исполнения, ускорение и эффективность

Длина текста байт	Число потоков	Время мс	Ускорение S	Эффективность %
100 000	1	0,434	1,000	100,00
100 000	2	0,360	1,206	60,28
100 000	4	0,337	1,288	32,20
100 000	6	0,270	1,607	26,79
100 000	8	0,242	1,793	22,42
100 000	12	0,226	1,920	16,00
100 000	32	0,333	1,303	4,07
100 000	128	1,046	0,415	0,32
100 000	1024	9,731	0,045	0,0044
1 000 000	1	4,539	1,000	100,00
1 000 000	2	3,564	1,274	63,68
1 000 000	4	2,656	1,709	42,72
1 000 000	6	2,038	2,227	37,12
1 000 000	8	1,493	3,040	38,00
1 000 000	12	1,635	2,776	23,13
1 000 000	32	1,489	3,048	9,53
1 000 000	128	1,716	2,645	2,07
1 000 000	1024	9,358	0,485	0,05
5 000 000	1	22,463	1,000	100,00
5 000 000	2	16,100	1,395	69,76
5 000 000	4	8,861	2,535	63,38
5 000 000	6	11,161	2,013	33,54
5 000 000	8	7,837	2,866	35,83
5 000 000	12	6,483	3,465	28,87
5 000 000	32	6,356	3,534	11,04
5 000 000	128	6,405	3,507	2,74
5 000 000	1024	10,942	2,053	0,20

Длина образца 16 байт. Для текста длиной 5 млн байт наименьшая медиана среди проверенных конфигураций получена при p = 32: 6,356 мс, ускорение 3,534.

Повторяющийся текст и несовпадение в конце образца

Таблица 4. Время исполнения, ускорение и эффективность

Длина текста байт	Число потоков	Время мс	Ускорение S	Эффективность %
100 000	1	3,469	1,000	100,00
100 000	2	1,973	1,758	87,91
100 000	4	1,010	3,435	85,87
100 000	6	0,744	4,663	77,71
100 000	8	0,783	4,430	55,38
100 000	12	0,689	5,035	41,96
100 000	32	0,617	5,622	17,57
100 000	128	1,227	2,827	2,21
100 000	1024	9,579	0,362	0,04
1 000 000	1	34,895	1,000	100,00
1 000 000	2	19,550	1,785	89,25
1 000 000	4	9,806	3,559	88,96
1 000 000	6	6,644	5,252	87,54
1 000 000	8	7,088	4,923	61,54
1 000 000	12	6,386	5,464	45,54
1 000 000	32	4,969	7,023	21,95
1 000 000	128	4,816	7,246	5,66
1 000 000	1024	11,498	3,035	0,30
5 000 000	1	171,440	1,000	100,00
5 000 000	2	87,798	1,953	97,63
5 000 000	4	48,998	3,499	87,47
5 000 000	6	32,755	5,234	87,23
5 000 000	8	24,566	6,979	87,23
5 000 000	12	24,631	6,960	58,00
5 000 000	32	22,949	7,470	23,35
5 000 000	128	20,443	8,386	6,55
5 000 000	1024	22,802	7,519	0,73

Длина образца 128 байт. Для текста длиной 5 млн байт наименьшая медиана среди проверенных конфигураций получена при p = 128: 20,443 мс, ускорение 8,386.

Графики времени и ускорения

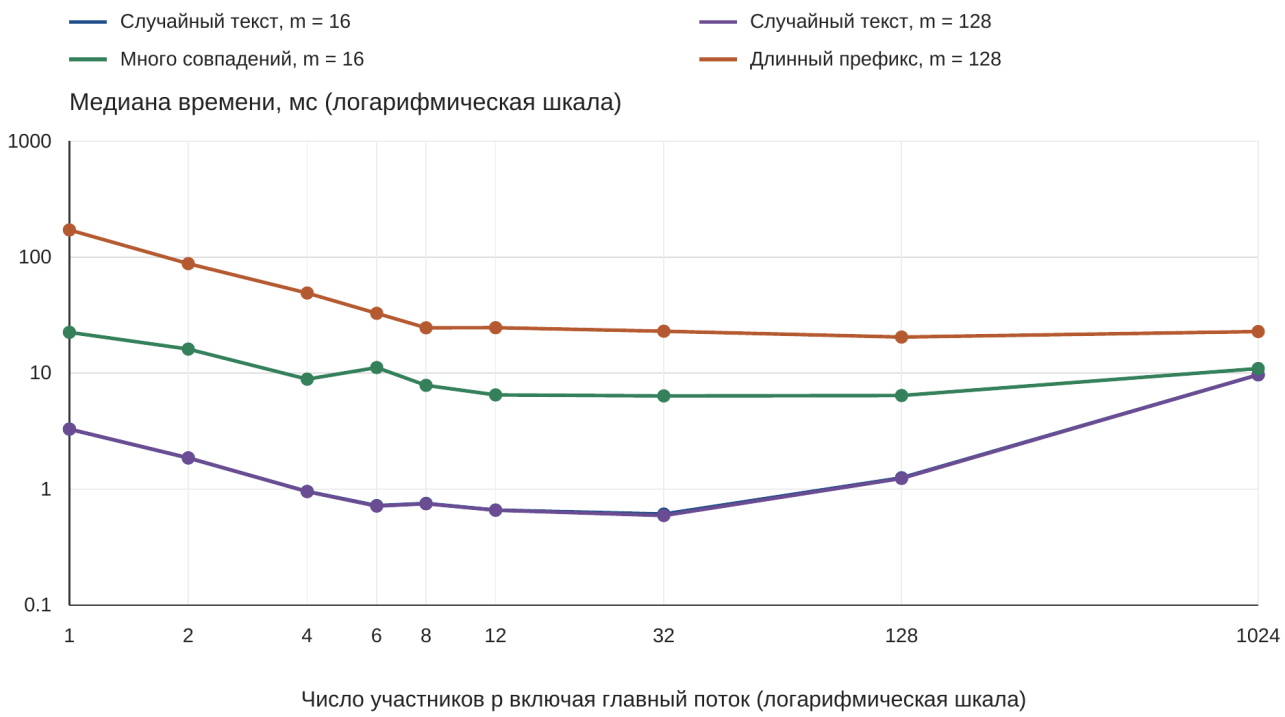


Рисунок 1. Медиана времени при длине текста 5 млн байт

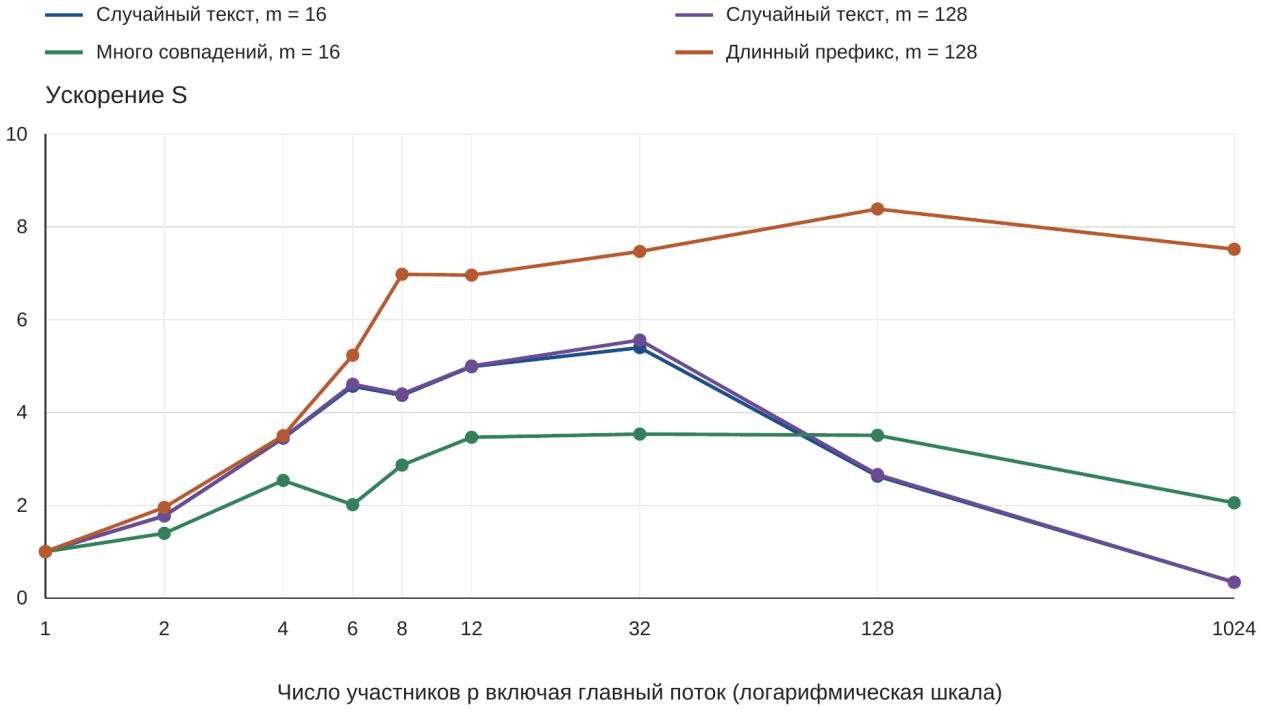


Рисунок 2. Ускорение при длине текста 5 млн байт

Графики эффективности и влияния размера входа

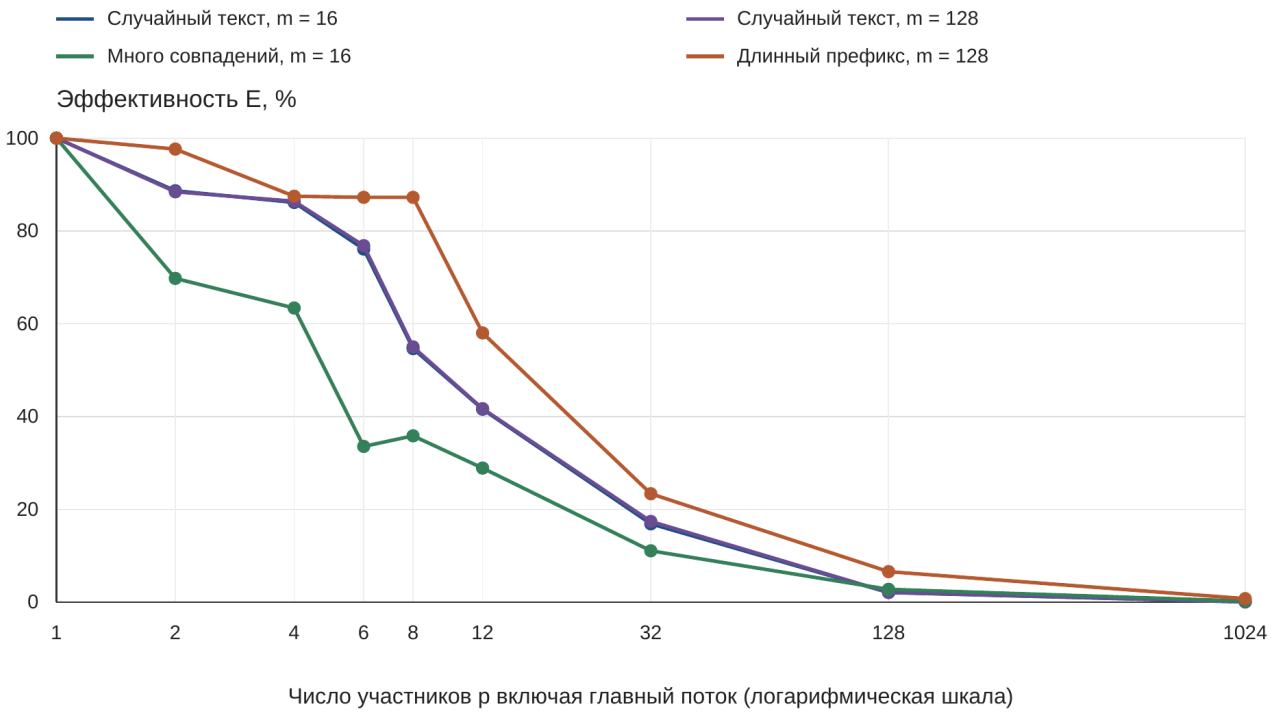


Рисунок 3. Эффективность при длине текста 5 млн байт

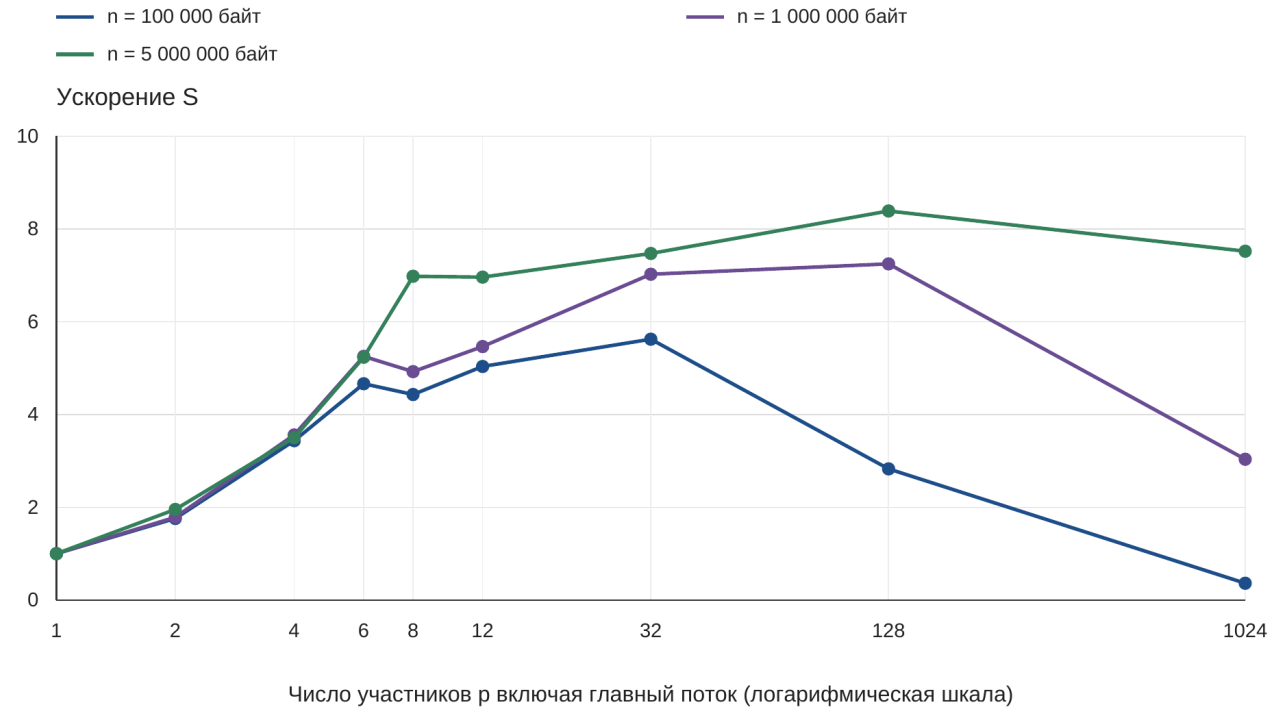


Рисунок 4. Ускорение тяжёлого случая при разных длинах текста

Анализ результатов

На коротком случайном тексте random_16 последовательная версия заняла 0,069 мс. При 1024 участниках получилось 9,853 мс, то есть примерно в 142,8 раза медленнее. Создание и ожидание большого числа потоков здесь обходятся дороже самого поиска.

Для случайного текста и образца длиной 16 байт при длине 5 млн байт последовательное время равно 3,287 мс. Лучшая медиана среди проверенных конфигураций получена при p = 32: 0,609 мс. Ускорение составило 5,397, эффективность - 16,87%.

Для текста с большим числом совпадений при длине 5 млн байт последовательное время равно 22,463 мс. Лучшая медиана среди проверенных конфигураций получена при p = 32: 6,356 мс. Ускорение составило 3,534, эффективность - 11,04%.

Для тяжёлого случая с образцом длиной 128 байт при длине 5 млн байт последовательное время равно 171,440 мс. Лучшая медиана среди проверенных конфигураций получена при p = 128: 20,443 мс. Ускорение составило 8,386, эффективность - 6,55%.

На случайном тексте большинство сравнений заканчивается на первых символах. На тексте из одинаковых букв приходится проверять весь образец. Поэтому время зависит не только от длины строки и образца, но и от их содержимого.

Число потоков сверх количества CPU иногда уменьшает измеренную медиану, но ускорение не растёт пропорционально числу потоков. Эффективность снижается из-за расходов запуска и ожидания, работы планировщика и ограниченного числа CPU. Отдельный вклад каждого фактора в этой работе не измеряется.

Число участников включает главный поток. Это не число потоков, которые исполняются на CPU одновременно: короткая задача может закончиться ещё до создания следующего потока. Результаты получены на одной машине, на синтетических входах, по пяти повторам; близкие значения могут меняться из-за фоновой нагрузки.

Вывод

В работе реализован наивный поиск образца в строке. При -t 1 поиск выполняется последовательно, а при большем значении позиции делятся между потоками POSIX Threads. Совпадения на границах участков не теряются. Количество потоков подтверждено командой ps.

Параллельная версия полезна, когда на поиск приходится достаточно много сравнений. Для тяжёлого случая длиной 5 млн байт получено ускорение 8,386 при 128 участниках и эффективность 6,55%. На коротких строках создание большого числа потоков может замедлять работу. Поэтому число потоков нужно выбирать с учётом размера и содержимого входных данных.